

CONST-051-tension

CONST-051(B) [?] CONST-051(C) Standalone-Build Tension

Status

RESOLVED via Option E (fork consolidation) — close-out⁴⁶, 2026-05-16.

Operator chose **Option E**. HelixAgent's 4 colliding-module replace directives migrated from `vasic-digital/*` → `HelixDevelopment/*` (the more-active canonical fork).
Verification: HelixAgent compile is green across `./internal/...`, `./cmd/helixagent/...`, and `./tests/integration/...` against the HelixDevelopment-side variants of DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine. The vasic-digital submodules remain on disk for the moment (deprecation sweep is a follow-up); HelixAgent no longer consumes them.

Task #274 (DebateOrchestrator) — the long-blocked sibling of this tension — was resolved in the same close-out by reconstruction from import surface; the new repo at `github.com/HelixDevelopment/DebateOrchestrator` is added as a submodule at `submodules/debate_orchestrator`.

Historical analysis preserved below for posterity. Original “operator decision required” section retained verbatim so the option-tradeoff reasoning remains discoverable.

(Historical) Surface after Tasks #254/#273

The tension

CONST-051(C) (nested-own-org chains forbidden) and CONST-051(B) (submodules-as-standalone, project-not-aware, reusable) pull in opposite directions for Go-module consumers like HelixAgent and HelixLLM.

Before Tasks #254 / #273 (CONST-051(C) violation)

```
// helix_agent/go.mod
replace digital.vasic.eventbus => ./EventBus
// helix_agent/.gitmodules
[submodule "EventBus"]
    path = EventBus
    url = git@github.com:vasic-digital/EventBus.git
```

Standalone-buildable (G5-style ✓), but violates G4 (nested own-org chain).

After Tasks #254 / #273 (CONST-051(B) violation)

```
// helix_agent/go.mod
replace digital.vasic.eventbus => ../submodules/event_bus
// helix_agent/.gitmodules
(no EventBus entry – moved to parent root)
```

Satisfies G4 ✓, but standalone clones break:

```
$ git clone git@github.com:HelixDevelopment/HelixAgent.git
$ cd HelixAgent
$ go build ./...
go: module digital.vasic.eventbus@v0.0.0 not found
```

Attempted fix: go.work workspace at HelixCode root

A go.work file with use ./HelixAgent + ./dependencies/vasic-digital/... was attempted (this round). It surfaced a SECOND conflict:

```
go: conflicting replacements for digital.vasic.docprocessor:
    ../submodules/doc_processor
    ../submodules/doc_processor
```

Both repos declare module digital.vasic.docprocessor even though they are DIFFERENT codebases under different upstream orgs. The workspace can't register both. Same conflict for: LLMOrchestrator, LLMProvider, VisionEngine (HelixDevelopment vs vasic-digital variants with shared module identifier).

go.work removed to restore HelixCode inner module's working compile state.

Additional evidence (close-out⁴⁵ Phase 1 deep)

Three findings reframe the option set:

Finding 1 — The 4 colliding modules are forks of a common ancestor, not independent codebases that happen to share a name. All four dependencies/{HelixDevelopment,vasic-digital}/{DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine} pairs share their first-commit SHA (DocProcessor root: 27888613...; LLMOrchestrator root: 91209efb...; LLMProvider root: 94fcea1f...; VisionEngine root: b2fcdcfb...). HelixDevelopment fork has more activity (DocProcessor: 63 vs 35 commits) — it is the more-maintained side.

Finding 2 — HelixCode does NOT depend on HelixAgent. grep helixagent helix_code/go.mod returns nothing. No source imports of HelixAgent module path. They are independent Go modules whose parent-build coexistence is incidental.

Finding 3 — The two consumers split on which fork they consume. - helix_code/go.mod → replace digital.vasic.docprocessor => ../submodules/doc_processor - helix_agent/go.mod → replace digital.vasic.docprocessor => ../submodules/doc_processor

The previous go.work attempt failed because it tried to register **both** sides under one workspace. But neither consumer needs both; each picks one fork.

Real-fix options (operator decision)

Option A: Module-path renaming

Distinguish the colliding modules by org-prefix in their module path:

```
// submodules/doc_processor/go.mod
module dev.helix.docprocessor // distinguish from
    digital.vasic.docprocessor
```

Requires upstream repo edits AND every consumer's go.mod update. Substantial coordinated work spanning two GitHub orgs.

Option B: Proper Go-module versioning

Publish each dependencies/vasic-digital/* module as a real Go module with semantic version tags. Consumers reference by version:

```
require digital.vasic.eventbus v1.0.0
// no `replace` needed
```

Caveat (close-out⁴⁵): B alone is INSUFFICIENT to resolve the collision. Two different VCS roots publishing the SAME module path (digital.vasic.docprocessor from both github.com/HelixDevelopment/... and github.com/vasic-digital/...) is still an unresolvable conflict for the Go module proxy — semver only disambiguates versions of one module path, not two repos sharing the same path. **B must be paired with A** to be correct.

Option C: Keep current state, accept CONST-051(B) gap

Document that HelixAgent + HelixLLM are NOT standalone-buildable post-#254/#273 by design — they're only consumable from HelixCode parent. This trades B↔C tension by sacrificing B for C. CONST-051(B) would need a note documenting "Go-module consumers operating under HelixCode parent are exempt from standalone-build expectation."

Option D: Hybrid — ship a bootstrap script

Add a scripts/bootstrap-standalone-build.sh to HelixAgent + HelixLLM that, on clone, clones the parent's dependencies/* tree adjacent and generates a temporary go.work pointing at them. Consumers can build standalone after running bootstrap once. CONST-054's helix-deps.yaml manifest is the natural metadata source for this script (avoids project-aware coupling per CONST-051(B)).

Option E: Fork consolidation (NEW — close-out⁴⁵)

Since the four collision pairs share ancestry (Finding 1), pick the more-maintained HelixDevelopment fork as canonical, deprecate the vasic-digital variants. HelixAgent migrates its 4 replace lines from vasic-digital/* to HelixDevelopment/*. Eliminates the collision permanently — there is only ONE module per identifier.

When E is the right answer: if the org-bifurcation has no business reason to persist (e.g. the duplicate forks were created for historical org-migration reasons that no longer apply).

When E is NOT the right answer: if the two forks have diverged in API or are owned/ maintained by different teams that need autonomy. In that case A+B is the right answer.

NOT-an-option F: Per-consumer go.work scope (REJECTED — close-out⁴⁵)

A previous draft of this doc considered scoping go.work per consumer (HelixCode root has its own; HelixAgent root has its own). This does NOT solve the standalone-build gap that CONST-051(B) demands — HelixAgent’s go.work would still reference ../dependencies/... paths that don’t exist after a fresh standalone clone. Per-consumer workspace only papers over the parent-build path that already works.

Recommendation (revised)

The option ranking depends on org policy:

If...	Pick
Both forks must persist with autonomy	A+B (rename + semver)
Forks can be consolidated	E (fork consolidation) — cheapest by far
Quick unblock without org coordination	D (bootstrap + helix-deps.yaml)
Defer until org policy is set	C (accept gap, document carve-out)

Default suggestion if no other input: E (fork consolidation). The shared-ancestry finding strongly suggests the bifurcation was incidental rather than intentional, and consolidation is the only option that requires no permanent ongoing maintenance burden (renames are a one-time event; semver tags need ongoing release discipline; bootstrap script needs ongoing maintenance; accept-the-gap means every new consumer hits the same surprise).

Audit trail

Date	Author	Notes
2026-05-15	Claude Opus 4.7	Tension surfaced during honest assessment after close-out ⁴¹ . Attempted go.work fix during close-out ⁴³ , removed after digital.vasic.docprocessor cross-org module-name collision broke HelixCode inner compile. Documented as architectural decision point.
2026-05-15	Claude Opus 4.7 (close-out ⁴⁵)	Phase 1 deep investigation: established forks-share-ancestry, HelixCode-doesn’t-depend-on-HelixAgent, original Option B is incomplete without A. Surfaced Option E (fork consolidation) and rejected Option F (per-consumer workspace). Revised recommendation to A+B / E / D / C ladder; default

Date	Author	Notes
		suggestion now E pending org-policy input.

Sources verified 2026-05-29: <https://go.dev/ref/mod> — Cross-referenced this doc’s Go-module claims against the latest official Go Modules Reference. Confirmed accurate as of this date: (a) the `replace` directive accepts a local relative path (`./` or `../`) to a replacement module root containing a `go.mod`, with the version omitted (the `replace` `digital.vasic.eventbus => ../dependencies/...` pattern in §“After Tasks #254/#273”); (b) two different VCS repositories declaring the SAME module path cannot both be resolved by the Go module system — if two or more modules in the build list provide a package the `go` command reports an error (the `digital.vasic.docprocessor` cross-org collision in §“Attempted fix” and Option B caveat); (c) `go.work` workspace files and the `use` directive (relative path to a module’s `go.mod`) behave as described in §“Attempted fix” and Option F, and the reference notes `go.work` is typically not committed to VCS. Negative finding: none — every module-system mechanism this doc relies on is current and accurate. Go version authority per CLAUDE.md §3.1: inner module `go 1.26`, root `go 1.25.2` (confirmed in-tree); no build/Docker version pins appear in this doc so none required correction.